

计算机视觉前沿技术报告： 线性神经网络

课程： 计算机视觉前沿技术 姓名： 牛晨勋
专业： 数据科学与大数据技术 学号： 8208231403

1 实验目标

基于二维带标签数据集，借助 Numpy 手搓实现线性二分类器 (softmax+ 交叉熵 + 梯度下降) 的完整训练流程，并对结果进行可视化。

2 实验内容及原理

2.1 实验内容

- 根据给定数据文件 (2 维数据)，进行线性二分类 (softmax+ 交叉熵 + 梯度下降)
- 用 Numpy 完成，不能使用 PyTorch、Tensorflow 等自动计算梯度的 python 库
- 记录并可视化训练过程的损失函数变化 (10 epochs)
- 可视化分类边界，展示不同训练阶段分类边界的变化过程 (可选，额外加分)
- 可以用 AI 辅助，同时提供 AI 代码 (学习)+ 自写代码 (不看 AI 代码后自己)

2.2 实验原理

a. 问题定义

给定一个二维数据集 $\{\mathbf{x}_i, y_i\}_{i=1}^N$ ，其中每条样本 $\mathbf{x}_i \in \mathbb{R}^2$ 对应一个类别标签 $y_i \in \{0, 1\}$ 。然后进行线性映射，将特征空间划分为两类区域，使模型对未见样本也能作出正确预测。



b. 数据预处理

为了平衡不同特征的尺度范围，为了加速梯度下降的收敛，我们对原始特征进行 Z-score 标准化：

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \quad j = 1, 2$$

- μ_j ：第 j 维特征的均值
- σ_j ：第 j 维特征的标准差。

标准化后，在特征矩阵最左侧拼接全为 1 的偏置列，使权重矩阵同时包含偏置项（线性模型里的截距），无需单独维护（我们把截距看作了一个额外特征的权重，而其特征全部是 1。由此，所有参数统一放在了一个权重矩阵中）：

$$\mathbf{X} = [\mathbf{1}, \tilde{\mathbf{X}}] \in \mathbb{R}^{N \times 3}$$

c. 线性模型

模型的线性层将输入映射为两个类别的得分（logit）：

$$\mathbf{Z} = \mathbf{XW}$$

$$1. \mathbf{W} = \begin{bmatrix} b^{(0)} & b^{(1)} \\ w_1^{(0)} & w_1^{(1)} \\ w_2^{(0)} & w_2^{(1)} \end{bmatrix}$$

- 第 0 列 $[b^{(0)}, w_1^{(0)}, w_2^{(0)}]^\top$ 是类别 0 的权重向量
- 第 1 列 $[b^{(1)}, w_1^{(1)}, w_2^{(1)}]^\top$ 是类别 1 的权重向量

$$2. \mathbf{z}_i = \mathbf{x}_i \mathbf{W} = [1, \tilde{x}_{i1}, \tilde{x}_{i2}] \cdot \begin{bmatrix} b^{(0)} & b^{(1)} \\ w_1^{(0)} & w_1^{(1)} \\ w_2^{(0)} & w_2^{(1)} \end{bmatrix}$$

- 其中第 i 行 $\mathbf{z}_i = [z_i^{(0)}, z_i^{(1)}]$ 分别是样本 $\mathbf{x}_i$ 属于类别 0 和类别 1 的原始得分

d. Softmax 激活函数

原始得分 $\mathbf{z}_i$ 的值域不受限制，可正可负可为零，而概率需要每个类别概率 ≥ 0 ，以及所有类别概率和 = 1。由此我们借助 Softmax 函数将其映射为合法的概率分布：



$$p_i^{(c)} = \frac{e^{z_i^{(c)}}}{e^{z_i^{(0)}} + e^{z_i^{(1)}}}$$

- **分子**： e^z 将任意实数映射为正数，且保持大小顺序（单调递增）。得分越高，指数越大，对应的概率也越大。
- **分母**：所有类别指数之和，进行归一化。
- 满足 $p_i^{(0)} + p_i^{(1)} = 1$ 且两者均为正数。

数值稳定操作：每行减去其最大值，避免 $e^{z_i^{(c)}}$ 上溢：

$$p_i^{(c)} = \frac{e^{z_i^{(c)} - MAX}}{\sum_{k \in \{0,1\}} e^{z_i^{(k)} - MAX}}, \quad c \in \{0,1\}$$

- 令 $MAX = \max\{e^{z_i^{(0)}}, e^{z_i^{(1)}}\}$

e. 交叉熵损失函数

由上一步可得模型预测的概率矩阵为 $\mathbf{P} \in \mathbb{R}^{N \times 2}$ ($\mathbf{p}_i = [p_i^{(0)}, p_i^{(1)}]$)，而对应的真实标签为 $\mathbf{y}$ ($y_i \in \{0,1\}$)，定义交叉熵损失为：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log p_i^{(y_i)}$$

- 求和后除以 N ：得到平均损失，使得损失值与 batch 大小无关，便于不同大小的数据集之间比较，也使得梯度尺度稳定，不随 batch 大小剧烈变化。
- 若模型对正确类别给出了高概率，则 $\log p_i^{(y_i)}$ 接近 0，损失小；反之损失大。

f. 梯度推导（反向传播）

设 $\mathbf{P}$ 为 $Softmax(\mathbf{Z})$ 输出 ($\mathbf{p}_i = [p_i^{(0)}, p_i^{(1)}]$)， $\mathbf{Y}_{oh} \in \mathbb{R}^{N \times 2}$ 为真实标签的 one-hot 编码矩阵 ($\mathbf{Y}_{oh} = [0,1]/[1,0]$ ，真实类别处为 1，其余为 0)。

可以推导出损失 $\mathcal{L}$ 对权重矩阵 $\mathbf{W}$ 的梯度：

第一步（对 logit 的梯度）：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}} = \frac{1}{N} (\mathbf{P} - \mathbf{Y}_{oh}) \in \mathbb{R}^{N \times 2}$$



第二步（对权重的梯度，由链式法则）：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} = \frac{1}{N} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y}_{\text{oh}}) \in \mathbb{R}^{3 \times 2}$$

g. 梯度下降更新规则

每轮进行迭代，更新权重：

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$$

- 梯度 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ 是一个向量，它指向当前点上升最快的方向，而我们想去最低点，即其相反的方向。
- 学习率 η 控制每次更新的步长：太小则收敛缓慢；太大则在最优解附近震荡，难以收敛。

h. 决策边界

训练完成后，对任意输入 $\mathbf{x}$ ，模型预测类别为：

$$\hat{y} = \arg \max_{c \in \{0,1\}} (W_{1_c} \cdot 1 + W_{2_c} \tilde{x}_1 + W_{3_c} \tilde{x}_2)$$

决策边界是两个类别得分相等的点集，即满足 $\mathbf{z}^{(0)} = \mathbf{z}^{(1)}$ 的二维特征空间中的一条直线。

3 实验代码

```
1 import numpy as np
2 from visualization import visualize_training
3 def load_data(path):
4     data = np.loadtxt(path, delimiter=',')
5     return data[:, :2], data[:, 2].astype(int)
6 def normalize_data(X):
7     mu = X.mean(axis=0)
8     sigma = X.std(axis=0)
9     return (X - mu) / sigma
10 def softmax(z):
11     z_shift = z - np.max(z, axis=1, keepdims=True)
```



```

12     exp_z = np.exp(z_shift)
13     return exp_z / np.sum(exp_z, axis=1, keepdims=True)
14 def cross_entropy(p, y):
15     n = len(y)
16     return -np.mean(np.log(p[np.arange(n), y] + 1e-12))
17 def gradient(X, y, p):
18     n, c = len(y), p.shape[1]
19     y_onehot = np.zeros((n, c))
20     y_onehot[np.arange(n), y] = 1
21     return X.T @ (p - y_onehot) / n
22 def train(path, epochs=10, lr=0.1):
23     X_raw, y = load_data(path)
24     X = normalize_data(X_raw)
25     X_bias = np.column_stack([np.ones(len(X)), X])
26     n_features, n_classes = X_bias.shape[1], 2
27     np.random.seed(0)
28     W = np.random.randn(n_features, n_classes) * 0.01
29     losses = []
30     weights = []
31     for i in range(epochs):
32         z = X_bias @ W
33         probs = softmax(z)
34         loss = cross_entropy(probs, y)
35         losses.append(loss)
36         weights.append(W.copy())
37         grad = gradient(X_bias, y, probs)
38         W = W - lr * grad
39     return X, y, losses, weights
40
41 X, y, losses, weights = train('./LinearNeuralNetwork/
    binary_classification.txt', epochs=10, lr=0.1)
42 visualize_training(X, y, losses, weights, 'loss_curve_numpy
    .png', 'decision_boundary_numpy.png', '[NumPyS] ')

```



4 实验结果

首先记录了每轮 epoch 结束时的损失值与准确率：

Epoch	Loss	Accuracy (%)
1	0.4673	83.00
2	0.4309	87.25
3	0.3997	91.00
4	0.3729	93.25
5	0.3496	94.25
6	0.3293	94.75
7	0.3114	95.50
8	0.2956	96.25
9	0.2815	96.25
10	0.2689	96.25

表 1: 各轮次训练损失与准确率

然后可视化了模型在 10 个 epoch 训练过程中交叉熵损失的变化曲线。(图中绿色标记为初始损失值，红色标记为第 10 轮结束时的损失值)

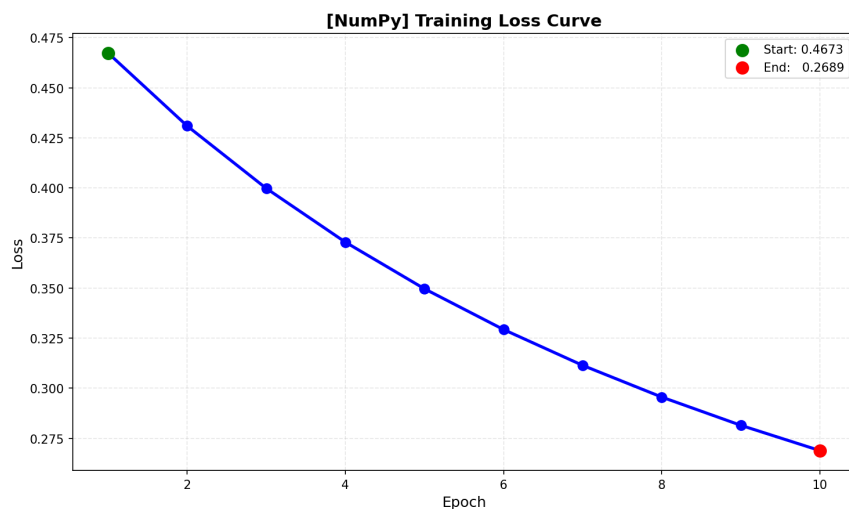


图 1: 训练损失曲线 (10 epochs, 学习率 $\eta = 0.1$)



接着是各训练阶段决策边界的变化过程。颜色由开始的浅蓝渐变至后续的深蓝，最终的黑色粗线为第 10 轮的边界。散点则是按真实类别着色（红色为类别 0，蓝色为类别 1）。

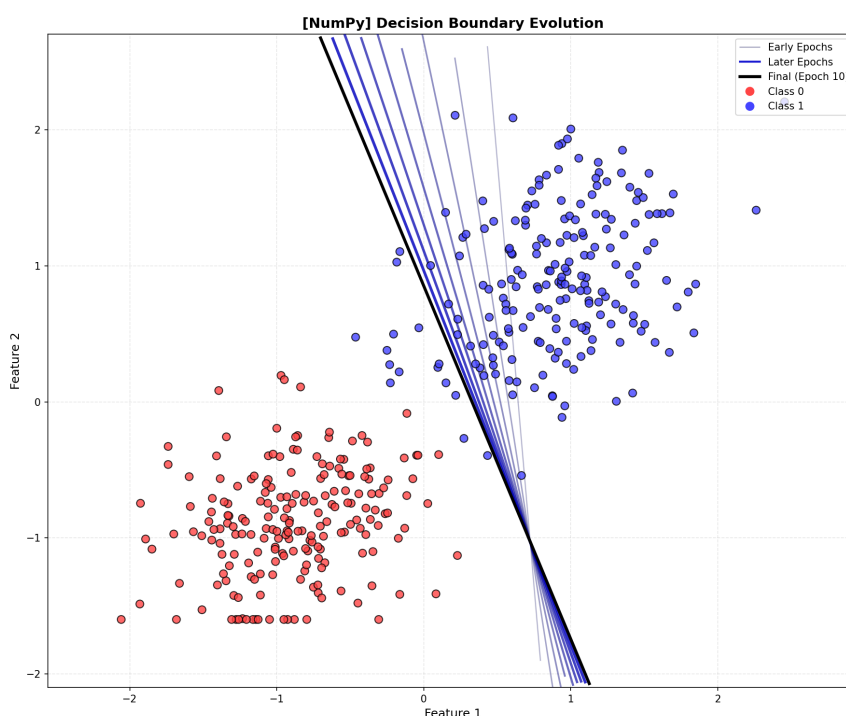


图 2: 决策边界演化图 (10 epochs, 浅色 → 深色 → 黑色)

5 实验分析

1. 初始边界（浅色细线）位置随机，几乎无法正确分隔两类样本，对应初始损失较高。

2. 前 5 个 epoch 损失下降较快（从 0.467 降至 0.350），后 5 个 epoch 下降幅度逐渐变缓（0.350 降至 0.269），符合梯度下降接近最优解时步长自然减小的特性。

3. 最终边界（黑色粗线）几乎分离了绝大多数样本，仅少数重叠区域的点被误分。最终边界位置与数据分布主轴一致，说明模型成功捕捉到了特征与类别间的线性关系。



6 附录

train_numpy.py

```
1 import numpy as np
2 from visualization import visualize_training
3 # 数据处理
4 def load_data(filepath):
5     """读取 CSV, 返回特征矩阵 X(N,2) 和标签 y(N,)"""
6     data = np.loadtxt(filepath, delimiter=',')
7     return data[:, :2], data[:, 2].astype(int)
8 def normalize(X):
9     """Z-score 标准化"""
10    mean, std = X.mean(axis=0), X.std(axis=0)
11    return (X - mean) / std, mean, std
12 def add_bias(X):
13    """拼接偏置列: (N,2) → (N,3)"""
14    return np.hstack([np.ones((len(X), 1)), X])
15 # 核心组件
16 def softmax(z):
17    """Softmax 激活函数"""
18    e = np.exp(z - z.max(axis=1, keepdims=True))
19    return e / e.sum(axis=1, keepdims=True)
20 def cross_entropy_loss(probs, y):
21    """多分类交叉熵损失"""
22    return -np.mean(np.log(probs[np.arange(len(y)), y] + 1e
23                        -15))
23 def compute_gradients(X, y, probs):
24    """反向传播梯度"""
25    N, C = len(y), probs.shape[1]
26    one_hot = np.zeros((N, C))
27    one_hot[np.arange(N), y] = 1.0
28    return X.T @ (probs - one_hot) / N
29 # 训练
30 def train_numpy(filepath, epochs=10, lr=0.1):
31    """纯 NumPy 线性分类器训练 (默认10 epochs) """
32    # 加载 & 预处理
```




```

33 X_raw, y = load_data(filepath)
34 print(f"\n[数据]  N={len(y)}, 类别0={np.sum(y == 0)},
      类别1={np.sum(y == 1)}")
35 X_norm, _, _ = normalize(X_raw)
36 X = add_bias(X_norm)  # (N, 3)
37 # Xavier 初始化
38 np.random.seed(42)
39 n_feat, n_cls = X.shape[1], 2
40 W = np.random.randn(n_feat, n_cls) * np.sqrt(2.0 /
      n_feat)
41 print(f"[模型]  W shape={W.shape} (含偏置行), Xavier
      初始化, seed=42")
42 print(f"[优化]  SGD, lr={lr}, epochs={epochs}")
43 loss_history = []
44 weights_history = []
45 for epoch in range(epochs):
46     # 前向传播
47     probs = softmax(X @ W)
48     loss = cross_entropy_loss(probs, y)
49     # 在参数更新前记录 (损失与权重对应同一时刻)
50     loss_history.append(loss)
51     weights_history.append(W.copy())
52     # 反向传播 + SGD 更新
53     W -= lr * compute_gradients(X, y, probs)
54     acc = np.mean(np.argmax(probs, axis=1) == y)
55     print(f"Epoch [{epoch + 1:2d}/{epochs}]  Loss: {
      loss:.6f}  Acc: {acc * 100:.2f}%")
56 print("-" * 60)
57 final_acc = np.mean(np.argmax(softmax(X @ W), axis=1)
      == y)
58 print(f"\n训练完成 | 最终 Loss={loss_history[-1]:.6f} |
      最终 Acc={final_acc * 100:.2f}%")
59 return X_norm, y, loss_history, weights_history
60 # 入口
61 if __name__ == "__main__":
62     DATA_FILE = './LinearNeuralNetwork/
      binary_classification.txt'

```



```

63     X_norm, y, loss_history, weights_history = train_numpy(
64         filepath=DATA_FILE,
65         epochs=10,    # + epochs
66         lr=0.1
67     )
68     visualize_training(
69         X_norm, y, loss_history, weights_history,
70         loss_filename='loss_curve_numpy.png',
71         boundary_filename='decision_boundary_numpy.png',
72         title_prefix='[NumPy] '
73     )

```

visualization.py

```

1  import os
2  os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
3  import numpy as np
4  import matplotlib
5  matplotlib.use('Agg')
6  import matplotlib.pyplot as plt
7  from matplotlib.colors import ListedColormap
8  from matplotlib.lines import Line2D
9  SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
10
11 def plot_loss_curve(loss_history, save_path, title='
    Training Loss Curve'):
12     """绘制损失曲线，标注起点（绿）和终点（红）"""
13     fig, ax = plt.subplots(figsize=(10, 6))
14     epochs = range(1, len(loss_history) + 1)
15     ax.plot(epochs, loss_history, 'b-', linewidth=2.5,
16             marker='o', markersize=8)
17     ax.scatter([1], [loss_history[0]], color='green', s
18               =100, zorder=5,
19               label=f'Start: {loss_history[0]:.4f}')
20     ax.scatter([len(loss_history)], [loss_history[-1]],
21               color='red', s=100, zorder=5,
22               label=f'End: {loss_history[-1]:.4f}')
23     ax.set_xlabel('Epoch', fontsize=12)

```



```

21 ax.set_ylabel('Loss', fontsize=12)
22 ax.set_title(title, fontsize=14, fontweight='bold')
23 ax.grid(True, alpha=0.3, linestyle='--')
24 ax.legend(fontsize=10)
25 plt.tight_layout()
26 plt.savefig(save_path, dpi=150, bbox_inches='tight')
27 plt.close()
28 print(f" 损失曲线已保存: {save_path}")
29
30 def plot_decision_boundary_evolution(X, y, weights_history,
31                                     save_path,
32                                     title='Decision
33                                     Boundary
34                                     Evolution'):
35     """将各 epoch 的决策边界叠加在同一图上，早期浅蓝、后期
36     深蓝、最终黑色粗线。"""
37     plt.figure(figsize=(12, 10))
38     x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
39     y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
40     x_line = np.linspace(x_min, x_max, 200)
41     plt.scatter(X[:, 0], X[:, 1], c=y,
42                cmap=ListedColormap(['#FF4444', '#4444FF'])
43                ,
44                edgecolors='k', s=60, alpha=0.8, zorder=5)
45     n = len(weights_history)
46     for i, W in enumerate(weights_history):
47         wd = W[:, 1] - W[:, 0]
48         if abs(wd[2]) < 1e-6:
49             continue
50         y_line = -(wd[0] + wd[1] * x_line) / wd[2]
51         mask = (y_line >= y_min) & (y_line <= y_max)
52         if not mask.any():
53             continue
54         t = i / (n - 1) if n > 1 else 1.0 # 0 → 1 随 epoch
55         递增
56         if i == n - 1: # 最终边界：粗黑线
57             plt.plot(x_line[mask], y_line[mask], 'k-',

```



```

        linewidth=3, zorder=4)
52     else: # 过渡边界: 蓝色渐深
53         plt.plot(x_line[mask], y_line[mask], '--',
54                 color=(0, 0, 0.3 + 0.5 * t),
55                 linewidth=1.0 + 2.0 * t,
56                 alpha=0.2 + 0.7 * t,
57                 zorder=3)
58     legend = [
59         Line2D([0], [0], color=(0, 0, 0.3), lw=1, alpha
60               =0.4, label='Early Epochs'),
61         Line2D([0], [0], color=(0, 0, 0.8), lw=2, alpha
62               =0.9, label='Later Epochs'),
63         Line2D([0], [0], color='black', lw=3, label=f'Final
64               (Epoch {n})'),
65         Line2D([0], [0], marker='o', color='w',
66               markerfacecolor='#FF4444',
67               markersize=10, linestyle='None', label='
68               Class 0'),
69         Line2D([0], [0], marker='o', color='w',
70               markerfacecolor='#4444FF',
71               markersize=10, linestyle='None', label='
72               Class 1'),
73     ]
74     plt.legend(handles=legend, loc='upper right', fontsize
75               =10)
76     plt.xlabel('Feature 1', fontsize=12)
77     plt.ylabel('Feature 2', fontsize=12)
78     plt.title(title, fontsize=14, fontweight='bold')
79     plt.xlim(x_min, x_max)
80     plt.ylim(y_min, y_max)
81     plt.grid(True, alpha=0.3, linestyle='--')
82     plt.tight_layout()
83     plt.savefig(save_path, dpi=150, bbox_inches='tight')
84     plt.close()
85     print(f" 决策边界图已保存: {save_path}")
86
87 def visualize_training(X, y, loss_history, weights_history,

```



```

80         loss_filename='loss_curve.png',
81         boundary_filename='
            decision_boundary.png',
82         title_prefix=''):
83     prefix = title_prefix.strip()
84     plot_loss_curve(
85         loss_history,
86         save_path=os.path.join(SCRIPT_DIR, loss_filename),
87         title=f'{prefix} Training Loss Curve'.strip()
88     )
89     plot_decision_boundary_evolution(
90         X, y, weights_history,
91         save_path=os.path.join(SCRIPT_DIR,
92                                 boundary_filename),
93         title=f'{prefix} Decision Boundary Evolution'.strip
            ()
    )

```

